Blockchain & Secure Multi-Party Computation

Master 2 — Computer Science

Session 4 of 12 — Cryptography for Blockchain II: Consensus Mechanisms

Where We Are

Session 3 covered the cryptographic primitives (hash functions, signatures, encryption, commitments, ZKPs) that secure individual transactions and data. This session covers the remaining critical component: **consensus mechanisms** — the protocols that allow a distributed network of potentially untrustworthy nodes to agree on a single, correct version of the blockchain. Without consensus, there is no shared ledger; without a secure consensus mechanism, the ledger can be corrupted by malicious nodes.

1 The Consensus Problem in Blockchain

Definition — Consensus in Distributed Systems

A consensus protocol achieves **agreement**: all honest nodes decide on the same value. It ensures **validity**: the decided value was proposed by some honest node (not fabricated). It guarantees **termination**: all honest nodes eventually decide. In blockchain: all honest nodes must agree on the same ledger state, the agreed ledger must only contain valid transactions, and the process must complete in finite time.

The central challenge is achieving consensus in the presence of **Byzantine faults** — nodes that behave arbitrarily (not just crash, but actively lie, send contradictory messages, or collude). This is fundamentally harder than tolerating simple crash failures.

2 Byzantine Fault Tolerance

Definition — Byzantine Generals Problem

Lamport, Shostak, and Pease (1982). A group of n generals surrounding a city must agree on a common battle plan (attack or retreat). They communicate only by messenger. Some generals may be traitors who send conflicting orders to different generals. The problem: design a messaging algorithm that guarantees loyal generals reach the same decision, even if up to f generals are traitors. The result: agreement is possible if and only if $n \geq 3f + 1$, i.e., traitors must be strictly fewer than one-third of all participants.

Example — Blockchain nodes as Byzantine generals

In a blockchain network, each node is a "general." Honest nodes follow the protocol. Malicious nodes may broadcast conflicting block proposals to different parts of the network, trying to fork the chain. The Byzantine Generals analogy is precise: the "city" is the transaction to be confirmed; each "general" is a validator node; each "battle plan" is a block proposal; "traitors" are adversarial or Sybil nodes. The BFT result tells us that a blockchain can tolerate up to $f < n/3$ malicious validators while still reaching consensus — a fundamental theoretical limit.

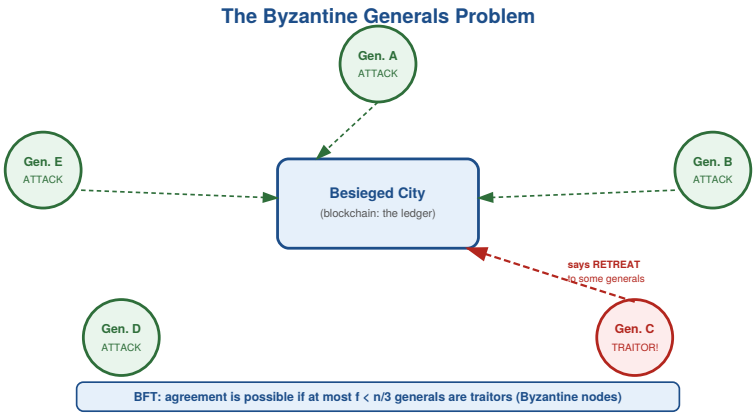


Figure 1: Four loyal generals and one traitor (Gen. C) who sends contradictory messages. With $n = 5$ and $f = 1$, since $5 \geq 3(1) + 1 = 4$, BFT consensus is achievable: the 4 loyal generals can still agree by majority.

? Think about it

Give an example from the real world.

✓ Answer

Practical Byzantine Fault Tolerant (PBFT) consensus, developed by Castro and Liskov in 1999, is used in permissioned blockchains like Hyperledger Fabric. In a consortium of 10 banks running PBFT, up to 3 banks could be compromised or act maliciously and the remaining 7 would still reach correct consensus. This is exactly the $n \geq 3f + 1$ limit: $10 \geq 3(3) + 1 = 10$.

2.1 The CAP Theorem and Blockchain

Definition — CAP Theorem

A distributed system cannot simultaneously guarantee all three of: **Consistency** (all nodes see the same data at the same time), **Availability** (every request receives a response), and **Partition Tolerance** (the system operates despite network splits). Blockchain networks prioritize Consistency and Partition Tolerance (CP systems), accepting that during network partitions, some nodes may temporarily be unavailable rather than serving inconsistent data.

? Think about it

Bitcoin sometimes has two valid competing blocks at the same height (a "fork"). Does this mean Bitcoin violates Consistency?

✓ Answer

Temporarily, yes — Bitcoin uses "eventual consistency." Two miners might find valid blocks simultaneously; both propagate and temporarily coexist (an "orphan" situation). The network resolves this by following the "longest chain rule": whichever branch accumulates more PoW first becomes the canonical chain. The shorter branch is abandoned. Consistency is restored, but not instantly. This is why 6 Bitcoin confirmations (roughly 1 hour) are typically required for large transactions: after 6 blocks on top of a transaction, rewriting the history including that transaction is computationally infeasible.

3 Proof of Work

Definition — Proof of Work (PoW)

Miners compete to find a nonce η such that:

$$H(\text{block header} \| \eta) < \text{target}$$

where H is SHA-256 (applied twice in Bitcoin), and *target* is a value automatically adjusted every 2016 blocks to maintain an average of one block per 10 minutes. The **difficulty** is the ratio $D = \text{max_target}/\text{target}$. The expected number of hash computations to find a valid nonce is $D \cdot 2^{32}$.

Example — Bitcoin's difficulty adjustment

Bitcoin's target is automatically recalculated every 2016 blocks (roughly 2 weeks). If the previous 2016 blocks were produced faster than 10 minutes each on average (because more miners joined), the difficulty increases (target decreases) to slow block production back to 10 minutes. If blocks were too slow (miners left), difficulty decreases. This self-regulating mechanism ensures predictable block timing regardless of how much total mining power participates.

Example — The energy cost of PoW

As of 2024, the Bitcoin network consumes approximately 120–150 TWh of electricity per year — comparable to the total electricity consumption of a country like Argentina or Poland. This energy expenditure is not a design flaw but a deliberate security mechanism: rewriting the blockchain requires redoing all this work, making historical revision prohibitively expensive. Critics argue that equivalent security could be achieved at a fraction of the energy cost; proponents argue that the energy expenditure's physical reality (unlike staked tokens) provides unique security properties.

? Think about it

Why can't Bitcoin simply switch from PoW to PoS to save energy?

✓ Answer

Technically, a majority of nodes would need to agree to change the consensus rules — a hard fork. But more fundamentally, Bitcoin's core community views PoW's energy expenditure as a feature (physical cost = security guarantee) rather than a bug. There is also significant resistance to change in a decentralized system with no central authority to mandate an upgrade. Ethereum successfully switched from PoW to PoS in September 2022 ("The Merge"), but Bitcoin's community has consistently rejected similar proposals.

4 Proof of Stake

Definition — Proof of Stake (PoS)

Validators lock up (stake) a quantity of cryptocurrency as collateral. A validator v_i with staked amount s_i is selected to propose the next block with probability proportional to their stake: $\Pr[\text{selected}] = s_i / \sum_j s_j$. If a validator behaves dishonestly (e.g., signs two conflicting blocks — "equivocation"), their stake is partially or fully destroyed (**slashing**). The economic loss from slashing is the security guarantee replacing PoW's computational cost.

Example — Ethereum's Proof of Stake (post-Merge)

After "The Merge" in September 2022, Ethereum switched from PoW to PoS. A validator must stake exactly 32 ETH to participate. A committee of validators (randomly selected from the full validator set) attests to each block, providing fast finality. The random selection uses RANDAO (a commit-reveal randomness scheme) combined with a VDF (Verifiable Delay Function) to prevent validators from manipulating selection outcomes. Ethereum's energy consumption dropped by 99.95% overnight.

? Think about it

In PoS, a wealthy entity that accumulates more stake gains more block proposals and therefore more rewards, allowing them to accumulate even more stake. Is this a problem?

✓ Answer

Yes, this is the "rich get richer" concern. Over time, PoS could lead to increasing centralization as wealthy validators compound their advantage. Different PoS systems address this differently: Ethereum caps individual validator influence through committee-based attestation; Cardano uses delegation to allow small holders to pool their stake; some systems introduce diminishing returns for very large stakes. It is an ongoing research and governance challenge with no universally agreed solution.

4.1 Delegated Proof of Stake (DPoS)

Definition — Delegated Proof of Stake (DPoS)

Token holders vote to elect a fixed number of delegates (typically 21–101), who take turns producing blocks in a round-robin schedule. The number of votes a delegate receives is proportional to the tokens delegated to them by the community. Delegates who miss their block production slot or behave dishonestly lose their delegate status at the next vote.

Example — EOS and DPoS

EOS uses DPoS with 21 elected block producers. This delivers extremely fast transaction throughput (thousands of TPS) because only 21 known entities need to reach consensus, unlike Bitcoin's thousands of competing miners. However, critics argue this represents a much weaker form of decentralization: just 21 entities effectively control the network, and there have been documented cases of block producers colluding to reverse transactions, something theoretically impossible in a sufficiently large PoW network.

? Think about it

DPoS processes thousands of transactions per second, while Bitcoin processes about 7. Why don't all blockchains use DPoS?

✓ Answer

Because there is a fundamental trade-off: throughput vs. decentralization. DPoS achieves speed by concentrating block production in a small number of elected validators — but this makes it much easier for a small group to collude, and introduces a democratic-like process for validator selection that can be captured by large token holders. Bitcoin's low TPS is the direct cost of having thousands of competing miners globally, which makes it vastly more resistant to capture or shutdown by any single actor. The right choice depends on the application's priority.

5 Proof of Authority

Definition — Proof of Authority (PoA)

In PoA, a predefined list of trusted, identified validators ("authorities") takes turns producing blocks. There is no cryptographic puzzle and no staking requirement. Security relies entirely on the reputation and legal accountability of the named validators. Any validator caught misbehaving can be removed from the authority list by the network governance.

Example — PoA in enterprise blockchains

VeChain (supply chain), BSC (Binance Smart Chain), and the Ethereum Ropsten test network have used PoA variants. In VeChain, 101 "Authority Masternodes" run by vetted companies and individuals (whose identities are publicly known and legally accountable) produce all blocks. This delivers extremely fast finality (single-second) at the cost of being entirely centralized: if the 101 authority nodes collude or are compelled by a government, the entire chain is under their control. This is acceptable for enterprise supply chain applications where the participants are already contractually bound.

6 Proof of Chance

Definition — Proof of Chance (PoC) — Original Proposal

Proof of Chance is a consensus mechanism in which the right to produce the next block is allocated through a **cryptographically verifiable random selection** independent of computational power and independent of stake size. Each eligible node generates a local random value using a Verifiable Random Function (VRF):

$$(\pi, \beta) = \mathrm{VRF}_{sk_i}(\text{seed})$$

where sk_i is the node's private key, *seed* is derived from the previous block hash (publicly known), π is a proof, and $\beta = H(\pi) \in [0, 1)$ is the random output. A node is eligible to produce the next block if $\beta < \theta$ (a threshold value set by network parameters). The node broadcasts its block along with the proof π, which any other node can verify in one step: $\mathrm{VRF.Verify}(pk_i, \text{seed}, \pi) = 1$ and $H(\pi) < \theta$.

Example — Why VRF makes PoC both random and verifiable

A plain random number (like a dice roll) could be faked: a node could claim to have rolled a "winning" number without evidence. A VRF solves this: the output β is uniquely determined by the node's private key sk_i and the public seed. The proof π allows any verifier to confirm that β was computed correctly from the legitimate sk_i, without the verifier needing to know sk_i. The randomness is unbiasable by the producing node (they cannot choose a different seed), unpredictable by outsiders (they cannot compute β without sk_i), and publicly verifiable.

? Think about it

What prevents a node from waiting — refusing to submit its VRF output — until it sees a favorable outcome?

✓ Answer

This is the "last revealer" or "grinding" attack. In PoC, the seed for each round is derived from the previous block's hash (or a committed value from the previous round), which is fixed before any node computes their VRF. The node cannot influence the seed, and the VRF output is uniquely determined by (seed, private key) — it cannot be "re-rolled." A node can choose not to reveal a winning output (at the cost of missing the block reward), but it cannot influence what the next seed will be either, since that depends on the next block's content, which requires consensus to produce. Careful protocol design with commitment phases can further reduce this attack surface.

? Think about it

How does PoC compare to PoW and PoS in terms of fairness and energy efficiency?

✓ Answer

PoC avoids PoW's energy waste entirely: VRF computation is negligible compared to SHA-256 mining. Unlike PoS, the probability of being selected does not depend on how much cryptocurrency a node holds — a node with 1 coin and a node with 1 million coins have the same chance per round (assuming both pass any minimum eligibility criterion). This makes block production more egalitarian. The security guarantee comes not from burnt energy (PoW) or financial stake (PoS) but from the cryptographic unforgeability of the VRF: a node cannot produce a valid proof for a block it was not legitimately selected to produce, making block forgery computationally infeasible under standard discrete logarithm assumptions.

7 Other Consensus Mechanisms

7.1 Practical Byzantine Fault Tolerance (PBFT)

PBFT, introduced by Castro and Liskov (1999), is a three-phase protocol (pre-prepare, prepare, commit) that achieves Byzantine fault tolerance among a known, fixed set of $n \geq 3f + 1$ nodes. It provides strong finality (once a block is committed, it is final — no forks) and operates without any puzzle or stake. The drawback is poor scalability: it requires $O(n^2)$ messages per consensus round, making it impractical for more than a few hundred nodes.

7.2 Proof of History (PoH)

Used by Solana, Proof of History is not a consensus mechanism by itself but a timekeeping mechanism: a node continuously evaluates a SHA-256 hash chain ($h_{i+1} = H(h_i, \text{data})$), creating a verifiable, cryptographically secure record of time passing. This allows validators to efficiently agree on the ordering of events without needing to coordinate timestamps explicitly, enabling Solana's extremely high transaction throughput (50,000+ TPS).

7.3 Nominated Proof of Stake (NPoS)

Used by Polkadot, NPoS separates the roles of validators (who produce blocks) and nominators (who stake tokens behind validators they trust). This creates a more democratic structure than simple PoS: small holders can participate by nominating trusted validators, and the election algorithm attempts to distribute stake evenly across validators to minimize concentration.

validators to minimize concentration.

7.4 The Nothing-at-Stake Problem and Slashing

Early PoS designs suffered from the "nothing-at-stake" problem: since voting for a fork costs nothing (unlike PoW where you can only mine one chain at a time), rational validators could simultaneously vote for all competing forks, undermining consensus.

Definition — Slashing in Proof of Stake

Slashing is the automatic destruction of a portion of a misbehaving validator's stake as a penalty. Common slashable offenses include: **equivocation** (signing two different blocks at the same block height) and **surround voting** (in Ethereum, creating attestations that contradict previous ones in a way that attempts to rewrite history). Slashing directly addresses the nothing-at-stake problem: dishonest behavior is now costly, mirroring PoW's energy expenditure cost.

Example — Ethereum slashing in practice

If an Ethereum validator signs two different blocks at the same slot (double voting), any other node that detects this can submit a "slashing proof" — the two conflicting signed messages — to a special smart contract. The slashing contract verifies the proof and confiscates a portion of the validator's 32 ETH stake (up to the entire stake in severe cases) plus forces the validator's exit from the validator set. The whistleblower node receives a small reward. This creates a self-policing incentive: validators watch each other for slashable behavior.

? Think about it

A validator's private key is compromised while the validator is on vacation. The attacker uses the key to perform equivocation, causing the validator to be slashed. What does this illustrate about PoS compared to PoW in terms of private key security?

✓ Answer

In PoW, a stolen private key loses the wallet balance but does not affect mining power (since mining power is hardware-based, not key-based). In PoS, the private key directly controls the staked collateral and is used for signing attestations. A compromised key can cause slashing, which destroys the staked ETH directly and irreversibly. This makes key security critically more important in PoS: hardware security modules (HSMs), remote signing setups, and careful key management are essential for large validators.

7.5 Nakamoto Consensus and the Longest Chain Rule

Definition — Nakamoto Consensus

Bitcoin's consensus model, informally called "Nakamoto Consensus," combines Proof of Work with the **longest chain rule**: if a node sees two competing chains of blocks, it always follows whichever chain has accumulated the most total Proof of Work (in practice, the longest chain). New blocks are always added to the longest known chain. This simple rule ensures convergence: honest nodes with the majority of hash power will always eventually produce the longest chain.

Example — Selfish mining attack on Nakamoto consensus

A miner with roughly 33% of total hash power can execute a "selfish mining" strategy: mine blocks privately without broadcasting them, then release them strategically to waste honest miners' work. This allows the selfish miner to earn a disproportionate share of block rewards

— more than their fair 33% — by exploiting the longest chain rule. At 50%+ hash power, an attacker can always override the honest chain. This attack, described by Eyal and Sirer (2014), is one reason mining pool concentration is a concern: large pools can approach the 33% threshold.

? Think about it

Does the longest chain rule guarantee that the chain cannot fork permanently?

✓ Answer

In theory yes, in practice near-certainly yes. As long as honest nodes control the majority of hash power, they will eventually produce a longer chain than any attacker, and all honest nodes will converge to that chain. A permanent fork would require an attacker to sustain more than 50% of hash power indefinitely while also convincing a significant portion of the ecosystem to follow the shorter chain — economically and practically infeasible at Bitcoin's scale. However, intentional community forks (like Bitcoin/Bitcoin Cash in 2017) can create permanent separate chains if different groups of users deliberately follow different rule sets.

Mechanism	Energy	Speed	Decentralization	Used by
PoW	Very high	Slow	High	Bitcoin, Monero
PoS	Very low	Medium-fast	Medium	Ethereum, Cardano
DPoS	Very low	Fast	Low (elected set)	EOS, TRON
PBFT	Very low	Fast	Low (small set)	Hyperledger
PoA	Very low	Very fast	Very low	VeChain, BSC
PoH	Very low	Very fast	Medium	Solana
PoC	Very low	Fast	High (egalitarian)	Research / novel

8 Finality: Probabilistic vs. Deterministic

A key dimension distinguishing consensus mechanisms is how they define "finality" — when a confirmed transaction can be considered irreversible.

Definition — Probabilistic vs. Deterministic Finality

Probabilistic finality (PoW, PoC): a transaction becomes exponentially harder to reverse as more blocks are built on top of it, but is never 100% final in theory. **Deterministic finality** (PBFT, PoA, and modern PoS with finality gadgets): once a block is committed through the consensus protocol, it is absolutely final and can never be reverted, even with unlimited computational resources.

Example — Why exchanges require 6 Bitcoin confirmations

Bitcoin's probabilistic finality means that a merchant accepting a transaction with zero confirmations (unconfirmed, still in the mempool) takes a significant risk: the transaction could be double-spent or evicted. With 1 confirmation (one block on top), the probability of reversal drops dramatically. With 6 confirmations (roughly 1 hour), rewriting the history

requires an attacker to produce 6 blocks faster than the rest of the network — requiring over 50% of global hash power sustained for 1 hour. Major exchanges typically require 3–6 Bitcoin confirmations for deposits, and 30+ for very large deposits.

? Think about it

Why would an application prefer deterministic finality even if it means sacrificing some decentralization?

✓ Answer

Many real-world applications — particularly financial ones — cannot accept probabilistic reversal risk, no matter how small. A stock exchange settling a million- dollar trade needs to know with certainty that the settlement is final, not "99.999% certain after 6 blocks." Interoperability protocols (Session 10) that move assets between blockchains also require deterministic finality on the source chain before they can safely release assets on the destination chain. For these applications, a consortium blockchain running PBFT (strong finality, limited decentralization) is preferable to Bitcoin's probabilistic model.

Session Summary

This session covered the consensus problem in blockchain, beginning with its formal foundations in the Byzantine Generals Problem ($n \geq 3f + 1$) and the CAP theorem. It then examined the main consensus mechanisms: Proof of Work (secure, energy- intensive, battle-tested), Proof of Stake (energy-efficient, economic security via slashing), Delegated Proof of Stake (fast, less decentralized), Proof of Authority (fast, centralized, enterprise-focused), Practical Byzantine Fault Tolerance (strong finality, limited scale), Proof of History (Solana's timekeeping innovation), and Proof of Chance (VRF-based random selection: egalitarian, energy-efficient, and cryptographically sound). The comparison table shows the fundamental trade-offs; no mechanism dominates all dimensions. Session 5 applies these foundations to the full internal structure of a blockchain.